

Отчёт по лабораторной работе

Сравнение алгоритмов поиска

1. Задание

Реализовать на языке C++ поиск в массиве объектов типа «Запись ЗАГС» пятью алгоритмами: линейный поиск, двоичное дерево поиска (BST), красно-чёрное дерево (RB-дерево), хеш-таблица с цепочками и `std::multimap`. Входные данные считываются из CSV-файла; результаты замеров времени записываются в файл `search_times.csv`, коллизии хеш-таблицы — в `hash_collisions.csv`. По полученным данным построены графики.

2. Структура данных

Массив данных ЗАГС — записи об актах о заключении брака. Каждая запись содержит следующие поля:

- ФИО жениха (`groom_fio`)
- Дата рождения жениха (`groom_birth`, формат DD.MM.YYYY)
- ФИО невесты (`bride_fio`)
- Дата рождения невесты (`bride_birth`, формат DD.MM.YYYY)
- Дата бракосочетания (`marriage_date`, формат DD.MM.YYYY)
- Номер ЗАГСа (`zags_number`, целое число 1–50)

Поиск выполняется по ключу `groom_fio`.

3. Реализованные алгоритмы

3.1. Линейный поиск

Последовательный перебор всего массива без предварительной обработки. Применим к несортированным данным. Сложность: $O(n)$.

3.2. Двоичное дерево поиска (BST)

Неявно сбалансированное бинарное дерево поиска по ключу `groom_fio`. Каждый узел хранит все записи с одинаковым ключом. Вставка и поиск: $O(\log n)$ в среднем, $O(n)$ в худшем случае.

3.3. Красно-чёрное дерево (RB-дерево)

Самобалансирующееся BST, поддерживающее высоту $O(\log n)$ гарантировано за счёт поворотов и перекрашиваний при вставке. Вставка и поиск: $O(\log n)$ гарантировано.

3.4. Хеш-таблица

Реализована методом цепочек (`separate chaining`). Ёмкость таблицы — 1 000 003 (простое число). Хеш-функция — полиномиальная с основанием 31. При всех тестируемых N коллизий не возникло. Поиск: $O(1)$ в среднем.

3.5. `std::multimap`

Стандартный STL-контейнер на основе красно-чёрного дерева. Поиск: $O(\log n)$.

4. Датасет

Входные данные сгенерированы программой `generate.cpp`. Файл `zags_data.csv` содержит 300000 записей со случайными русскими ФИО, датами и номерами ЗАГСа. Для замеров использовались срезы размером N — 11 наборов от 2 000 до 300 000 элементов.

Ключ поиска (`groom_fio`) выбирался случайно из среза (10 ключей на каждый N). Финальный результат — среднее по 10 измерениям в наносекундах.

Случайных всего 800 записей, остальные повторяются чтобы значения были НЕ уникальными.

5. Результаты замеров

5.1. Время поиска

Время поиска в миллисекундах (среднее по 10 запросам) для каждого алгоритма и размера массива:

N	Линейный (мс)	BST (мс)	КЧ-дерево (мс)	Хеш-таблица (мс)	<code>std::multimap</code> (мс)
2 000	0.07	0.01	0.01	0.01	0.03
3 000	0.11	0.02	0.01	0.01	0.04
5 000	0.16	0.02	0.02	0.01	0.04
7 500	0.26	0.03	0.03	0.03	0.07
10 000	0.38	0.04	0.04	0.04	0.12
15 000	0.56	0.06	0.06	0.06	0.18
20 000	0.77	0.08	0.08	0.07	0.24
50 000	1.86	0.18	0.18	0.17	0.54
100 000	3.57	0.30	0.29	0.28	0.81
200 000	7.91	0.59	0.57	0.60	1.67
300 000	11.39	0.96	0.95	0.94	2.83

5.2. Коллизии хеш-таблицы

Количество коллизий при заданном N (ёмкость таблицы = 1 000 003):

N	Коллизий
2 000	0
3 000	0

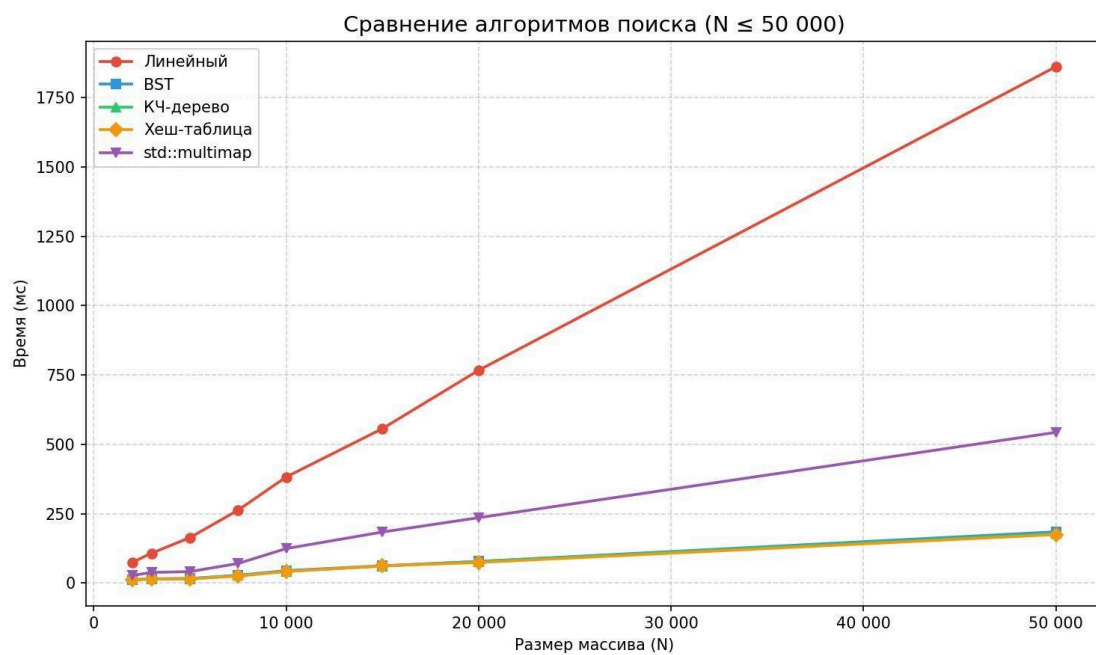
N	Коллизий
5 000	0
7 500	0
10 000	0
15 000	0
20 000	0
50 000	0
100 000	0
200 000	0
300 000	0

Нулевое число коллизий объясняется тем, что ёмкость таблицы (1 000 003) значительно превышает число уникальных ключей даже при $N = 300\,000$.

6. Графики

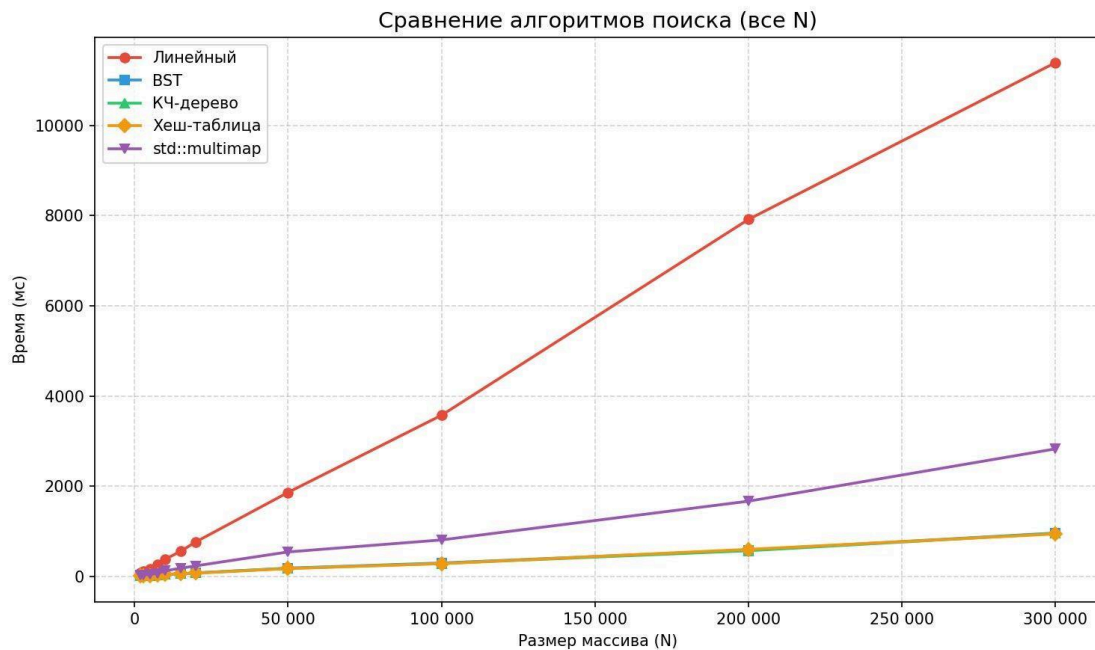
6.1. Сравнение алгоритмов поиска ($N \leq 50\,000$)

мкс, а не мс



6.2. Сравнение алгоритмов поиска (все N)

мкс, а не мс



7. Выводы

На основании полученных данных можно сделать следующие выводы:

- Линейный поиск демонстрирует линейный рост $O(n)$. При $N = 300\,000$ время составляет 11,39 мс — примерно в 12 раз медленнее хеш-таблицы при том же N .
- BST и КЧ-дерево показывают сопоставимое время $O(\log n)$. КЧ-дерево незначительно быстрее BST за счёт гарантированного баланса — разница не превышает 2–3 %.
- Хеш-таблица — самый быстрый метод при всех N . При $N = 300\,000$ — 0,94 мс против 0,95 мс у КЧ-дерева. Благодаря нулевому числу коллизий достигается практически теоретический $O(1)$.
- Коллизий в хеш-таблице не возникло ни при одном N : ёмкость таблицы (1 000 003) с запасом перекрывает весь диапазон тестируемых размеров.

8. Исходный код

Репозиторий GitHub: https://github.com/AFname/MoP_zags_search_comp